

Use Inference of Artificial Intelligence

The Pilot Model: Why the Value of AI Is Inferential and Its Risk Is Operational

Joshua Idele

Independent Researcher

codewithjoshh@gmail.com

GitHub: Coding-With-Josh X: @josh_scriptz

Preprint v3, October 2026

Abstract

Public and technical debate about artificial intelligence (AI) often sorts systems into "safe" and "dangerous." We argue that this framing misplaces the locus of risk and of value. Contemporary AI systems, including large language models (LLMs) and agentic tools, produce *inferences*: probabilistic outputs conditioned on training data, context, and instructions. Their benefit is inferential, arising when a human supplies intent and context and then evaluates what is returned. We propose the **Pilot Model**, in which the user is the pilot, the AI is an instrument, and safety is a property of the whole pilot-instrument-environment system rather than of the instrument alone. We give a simple expected-harm decomposition in which the operator controls three levers: the probability of error, the probability that an error is detected, and the impact of an undetected error. We distinguish four failure modes (context starvation, the oracle fallacy, unverified autonomy, and upstream misalignment), instantiate the model in software engineering as a validation architecture, show how it transfers to other domains, and propose an empirical protocol to test it. We also address the strongest objections, including that some systems are harmful by design, state where operator control runs out (harm without malicious intent, harm to third parties), and compare the model with existing safety approaches. Our claim is deliberately scoped: danger is not an intrinsic property of inference, but of deployments in which inference is given authority without verification, or is built for harmful ends.

Keywords: AI inference; human-AI collaboration; human oversight; automation bias; verification; software engineering

1. Introduction

When an AI system produces a wrong or harmful result, the common verdict is that the AI was "bad." This verdict attributes competence and intent to a system that has neither. A language model computes a probability distribution over possible continuations and samples from it; what it returns is an inference whose quality depends on the evidence, framing, and checking around it. Safety research has long recognized that accidents in complex systems rarely have a single component as their cause [1, 2]. We apply this view to modern AI.

This paper makes five contributions:

- **A thesis.** The value of AI is inferential: it generates plausible candidates (code, text, plans, hypotheses) quickly, but it is not an authority on truth.
- **A model.** The Pilot Model, with an expected-harm decomposition that identifies what an operator controls (Section 3).
- **A taxonomy.** Four failure modes in the human-AI loop (Section 4).

- **Limits and a comparison.** Where operator control runs out, including harm without malicious intent, and how the model relates to existing AI safety approaches (Sections 5-6).
- **An instantiation and a test.** A validation architecture for software engineering, its transfer to other domains, and a proposed evaluation protocol (Sections 7-9).

We scope the thesis carefully. We do not claim that no AI system can contribute to harm. We claim that danger is not an intrinsic property of inference; it is a property of how systems are built, incentivized, deployed, and supervised. The productive question therefore shifts from "Is this AI dangerous?" to "Who is piloting, with what instruments, checks, and limits on authority?"

Note on terminology. On September 29, 2026, a U.S. executive order directed federal agencies to replace "artificial intelligence" with "super intelligence" (SI) in executive-branch communications [16]. This paper retains "artificial intelligence" and "AI" for two reasons. It is the term used across the literature and standards this work builds on, and "superintelligence" conventionally denotes intellect that surpasses human capability [17], whereas this paper concerns systems that perform fallible, probabilistic inference. That distinction is central to the argument.

2. Background and Related Work

2.1 Code generation and evaluation

Chen et al. introduced HumanEval and the Codex models, reporting that functional correctness improves substantially with repeated sampling and that automated testing is necessary to separate correct from incorrect candidates [3]. SWE-bench moved evaluation to real repository issues and showed that realistic software tasks remain difficult without strong scaffolding [4]. These results frame generated code as candidates that must be checked.

2.2 Security of generated output

Pearce et al. found that a substantial fraction of Copilot-generated programs in security-relevant scenarios contained vulnerabilities [5]. In a user study, Perry et al. reported that participants with an AI assistant wrote less secure code than those without, while being more confident in its security [6]. The second finding is important for our argument: the failure is jointly produced by model output and human over-trust.

2.3 Automation, trust, and human error

The human-factors literature distinguishes appropriate use of automation from misuse (over-reliance) and disuse (under-reliance) [7], and documents complacency and automation bias when operators fail to cross-check automated aids [8]. Reason's analysis of human error [2] and Leveson's systems-theoretic view of safety [1] treat accidents as failures of control across a socio-technical system, which is the stance we adopt.

2.4 AI safety and model-level concerns

Work on AI safety identifies model-level problems such as reward misspecification, distributional shift, and unsafe exploration [9], and scholarship on large language models highlights risks from training data, scale, and misuse [10]. We take these seriously. Our position is that they are best addressed as properties of the *system* that includes builders, data, incentives, and operators, and that operator-side verification remains necessary even as model-side work improves.

3. The Pilot Model

3.1 Inference, not authority

LLM inference is probabilistic. It can be made reproducible (fixed seed, low temperature), but reproducibility is not correctness. Where determinism is required, it comes from the surrounding layer: compilers, type systems, tests, invariants, and human review. We therefore separate *generation* (probabilistic, fast, broad) from *validation* (deterministic or accountable, slower, narrow).

3.2 Roles

Dimension	Pilot (human)	Instrument (AI)
Core process	Intent, judgment, domain experience	Probabilistic pattern inference over supplied context
Strengths	Goals, tradeoffs, architecture, accountability	Speed, breadth of recall, drafting, variation
Limitations	Fatigue, cognitive load, slips and oversights	No grounded understanding; fluent but possibly wrong output
Responsibility	Owens the decision and its outcome	Carries none; extends the pilot's reach

Table 1. Complementary roles in the Pilot Model.

3.3 An expected-harm decomposition

Consider a single AI-assisted action. Let e be the probability that the output is wrong in a way that matters, d the probability that verification detects such an error before it takes effect, and I the impact if an error takes effect. The expected harm of the action is

$$E[\text{harm}] = e \times (1 - d) \times I$$

This is an illustrative decomposition, not an empirical law. It is useful because it shows where the operator has leverage:

- **Reduce e by supplying context.** Precise requirements, interfaces, constraints, and examples ground the inference.
- **Raise d by verifying.** Tests, static analysis, cross-checks, and independent review. Over-trust lowers effective d , which is the mechanism of automation bias [7, 8].
- **Bound I by limiting authority.** Sandboxes, least privilege, staged rollout, rollback, and human sign-off for irreversible actions.

Model builders influence the baseline e and supply tools that make d easier to achieve (uncertainty signals, provenance, safe defaults). A better model lowers e but does not remove the need for d and a bound on I . As autonomy increases, I tends to grow because more actions occur without a human between inference and effect, which increases rather than reduces the importance of the other two levers.

3.4 Why a pilot

Aviation illustrates the point. Instruments and autopilots greatly increase capability and safety, yet accidents still occur when instruments are poorly designed, when pilots over-trust or misread them, or when procedures fail. The lesson is not that instruments are dangerous but that safety is engineered across the whole system, including the pilot's authority to override. The metaphor has limits (Sections 5 and 10), but it captures the division of labor we propose.

3.5 Operationalizing the levers

A model is only useful if its terms can be measured. Table 2 gives, for each lever, a concrete example from practice and a measurable proxy that can be estimated in the evaluation of Section 9.

Lever	In practice (example)	How to measure it
Context (lowers e)	Giving a coding assistant the relevant interfaces, constraints, and error logs instead of a one-line request; giving a clinical decision-support tool the full patient history.	Error rate on identical tasks under minimal versus enriched context. The difference estimates the effect of context on e .
Verification (raises d)	A test suite, a security scan, or an independent second reviewer checking the output before it is used.	Seeded-defect detection: insert known defects into outputs and measure the share caught before release. This estimates d directly.
Limits (bounds I)	A sandbox, least-privilege permissions, staged rollout, and human sign-off for irreversible actions.	Blast radius of an undetected error: severity-weighted count of affected records or systems, reversibility, and time to roll back.

Table 2. Operational definitions and measurable proxies for the three levers.

Because the three quantities can be estimated separately, the model makes a testable prediction: interventions on different levers should have different effects on different kinds of tasks (for example, verification should matter most where errors are subtle, and limits where errors are irreversible). Ablating one lever at a time, as proposed in Section 9, tests this.

4. Failure Modes in the Human-AI Loop

Failure mode	Description	Lever affected
F1. Context starvation	Too little information is supplied; the model infers from generic priors and produces output that is plausible but wrong for this situation.	e (too high)
F2. The oracle fallacy	Output is treated as authoritative because it is fluent; checking is skipped or superficial.	d (too low)
F3. Unverified autonomy	The system acts (executes, merges, sends, deploys) without a gate between inference and effect.	I (unbounded)
F4. Upstream misalignment	Training data, objectives, or incentives favor speed or engagement over reliability and safety, or the system is built for harmful ends.	Baseline e ; designer responsibility

Table 3. A taxonomy of failures mapped to the model's levers.

F1-F3 are operator-side and are addressable by practice. F4 is builder- and institution-side and is addressable by design, standards, evaluation, and governance. Both are system-level causes; neither requires attributing intent to the model. This is also why accountability belongs to people and organizations rather than to the instrument.

5. Harm Without Malice: Where Operator Control Runs Out

The Pilot Model does not claim that harm requires bad intent, or that a diligent operator can always prevent it. AI systems can produce harmful results when nobody intends harm, and the levers in Section 3 have limits. We identify five.

- **Model-side error and misspecification.** Models can produce confident falsehoods, and optimizing a stated objective can yield behavior its designers did not intend [9]. This sets a baseline e that the operator does not control.

- **Data-driven bias.** Training data can encode and reproduce social biases and other harmful patterns [10]. Outputs can be harmful even when the operator acts in good faith.
- **Verification limits.** Some outputs cannot be checked cheaply by a person, such as novel scientific claims or large volumes of automated decisions. Here d is bounded no matter how diligent the operator is.
- **Third parties.** The people harmed are often not the pilot. A screening or lending tool affects people who never chose to use it and who cannot verify, override, or contest its output.
- **Speed and scale.** Automated systems can act faster, and on more cases, than oversight can follow, which raises I before a human can intervene.

These cases show that the Pilot Model is necessary but not sufficient. They also sharpen the thesis. Danger is not a property of inference as such, but it can be a property of a deployed system whose baseline error, verification limits, and reach exceed what any single operator controls. In those cases responsibility moves to builders, deployers, and regulators, and the relevant tools are model-side alignment and evaluation, institutional oversight, and rules about where such systems may be used (Section 6).

6. Comparison with Existing Approaches

The Pilot Model sits alongside, not in place of, existing approaches to AI safety. Table 4 summarizes how each relates to the three levers.

Approach	Primary target	Relation to the Pilot Model
Alignment training (RLHF [11], Constitutional AI [12])	Model behavior; baseline e (builder-side)	Complementary. Lowers baseline e before the operator acts, but does not remove the need for d and a bound on I .
Red teaming and evaluation [13], benchmarks [3, 4]	Discovering failure modes; estimating e	Supplies estimates of e and failure types that tell operators where to verify.
Risk management frameworks (NIST AI RMF [14])	Organizational governance across the AI lifecycle	Institutional layer. The Pilot Model offers an operator-level, measurable decomposition within it.
Human oversight requirements (EU AI Act, Art. 14 [15])	Legal requirement that high-risk systems permit effective human oversight	Mandates the pilot's role. The Pilot Model specifies what makes oversight effective (verification and bounded authority) and notes that oversight degrades under automation bias [7, 8].
Layered defense and systems safety [1, 2]	Barriers and control structures across a socio-technical system	Closest intellectual ancestor. The Pilot Model applies the idea to AI-assisted work through three explicit levers.

Table 4. Existing safety approaches and their relation to the three levers.

We make three claims about what is different. First, the model is *operator-centered*: it starts from what the person using the system can control rather than from properties of the model. Second, it separates context, verification, and authority in one decomposition, so interventions can be compared and ablated. Third, it comes with measurable proxies and a test protocol (Table 2 and Section 9). We do not claim these ideas are individually new. The contribution is their combination and operationalization for AI-assisted work, and the claim stands or falls on the evaluation.

7. Instantiation in Software Engineering

Software engineering is a favorable setting for the model because correctness is often mechanically checkable. The following architecture maps the levers to engineering practice.

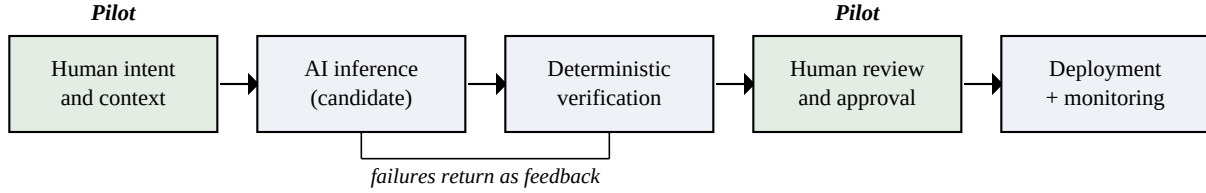


Figure 1. Pilot-centered validation architecture. Green boxes are human-controlled stages.

- **Contextual enrichment (lowers *e*).** Provide relevant files, interfaces, constraints, and error logs; structural representations such as abstract syntax trees can further ground generation.
- **Decomposed prompting (lowers *e*, eases *d*).** Break work into small, checkable steps rather than requesting a monolithic solution.
- **Automated guardrails (raises *d*).** Unit and property-based tests, type checking, linters, and security scanners catch statistical errors that review alone may miss.
- **Scoped autonomy (bounds *I*).** Agents receive least-privilege permissions, run in sandboxes, and require human approval for irreversible operations, with audit logs and rollback.

The architecture positions the engineer as an *engineer of context and verification*: the skill that matters is less typing code and more specifying intent, constructing checks, and judging results.

8. Generality Beyond Software

The three levers are not specific to code. The difference across domains is how easy it is to build a high-*d* verification step, which determines how much authority an AI-assisted action can safely carry.

Domain	Typical inference output	Verification (raises <i>d</i>)	Authority limit (bounds <i>I</i>)
Software	Code, tests, refactors	Compilers, tests, scanners, code review	Sandbox; staged deploy; approval for irreversible actions
Medicine	Differential suggestions, summaries	Clinician review; guideline cross-check	Decision support only; clinician signs the decision
Law	Research, draft clauses	Citation checking; attorney review	Drafts only; attorney of record files and advises
Education	Explanations, feedback	Teacher or source verification	Formative use; humans assign grades
Finance	Analyses, scenarios	Independent model validation; reconciliation	Position and loss limits; human approval above thresholds

Table 5. The levers across domains. Entries are illustrative of practice, not prescriptions.

Domains where verification is cheap (software, arithmetic, formal checking) can support greater delegation. Domains where verification is expensive or slow (open-ended judgment, novel clinical situations) warrant tighter limits on authority and more human involvement. The model thus yields a practical rule: *delegation should scale with verifiability, and authority should be bounded by the cost of an undetected error.*

9. Proposed Evaluation

To move from argument to evidence we propose a controlled comparison on a fixed set of tasks, with participants randomized to one of three conditions:

Condition	Description
A (baseline)	No AI assistance.
B (naive use)	AI assistance; output accepted after minimal review; no structured context.
C (pilot practice)	AI assistance with enriched context, decomposed tasks, automated guardrails, and scoped authority.

Table 6. Experimental conditions.

Measures: defect rate, security findings, time to completion, and reviewer-rated maintainability; for participant behavior, self-reported confidence versus measured correctness (calibration). **Hypotheses:** (H1) C yields fewer defects and security findings than B; (H2) C completes faster than A at comparable quality; (H3) B shows worse confidence calibration than C. **Analysis:** mixed-effects models with task and participant as random effects, with pre-registered hypotheses and effect sizes reported with confidence intervals. *[Results to be added after the study.]* A follow-up ablation removes one lever at a time from condition C to test which contributes most, and the proxies in Table 2 give each lever a measurable outcome. The same design can be adapted to other domains in Section 8 by substituting domain-appropriate tasks and error measures.

10. Discussion: Objections and Limitations

"Some AI is dangerous by design." Agreed. A system built to deceive, surveil, or harm is a harmful deployment, and responsibility lies with its designers and operators. This is failure mode F4, not a counterexample to the thesis. The thesis concerns inference as such and does not excuse any application.

"Bias and insecure patterns live in the model, not the user." Partly true, and it is why we include builders within the system analyzed. Model-level work on data, alignment, and evaluation [9, 10] is complementary (Sections 5 and 6). Even so, the operator cannot delegate verification to the instrument whose reliability is in question.

"Users cannot verify everything." Correct. This is why verification must be partly automated and why tool builders should expose uncertainty, provenance, and safe defaults. A pilot needs good instruments, and the model assigns that obligation to designers.

"The pilot metaphor weakens as autonomy grows." This is a genuine limit. Greater autonomy raises I , so it increases the demand for oversight mechanisms such as scoped permissions, monitoring, and interruption capability. Where meaningful human oversight is infeasible at the required speed or scale, the model advises against granting that authority.

Limitations. The framework is conceptual and the expected-harm decomposition is illustrative; e , d , and I are not independent in practice (for example, over-trust couples e and d). The domain analysis in Section 8 is a sketch requiring domain expertise. The proposed study has not yet been run, and rapid model progress may shift which failure modes dominate.

11. Conclusion

Asking whether an AI is safe or dangerous obscures where outcomes are decided. Inference supplies candidates; people supply direction, context, and judgment; engineering supplies verification and limits. Value emerges when the user pilots with good instruments and appropriate checks. Harm emerges when authority is granted to inference without verification, or when systems are built for harmful ends. For practitioners, the implication is to invest in context, verification, and bounded authority. For builders and

institutions, it is to make verification easy and incentives sound. For researchers, it is to test these claims empirically across domains.

References

- [1] N. G. Leveson, *Engineering a Safer World: Systems Thinking Applied to Safety*. MIT Press, 2011.
- [2] J. Reason, *Human Error*. Cambridge University Press, 1990.
- [3] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv:2107.03374, 2021.
- [4] C. E. Jimenez et al., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" in *Proc. ICLR*, 2024.
- [5] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," in *Proc. IEEE Symposium on Security and Privacy*, 2022.
- [6] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?" in *Proc. ACM CCS*, 2023.
- [7] R. Parasuraman and V. Riley, "Humans and Automation: Use, Misuse, Disuse, Abuse," *Human Factors*, vol. 39, no. 2, 1997.
- [8] R. Parasuraman and D. H. Manzey, "Complacency and Bias in Human Use of Automation: An Attentional Integration," *Human Factors*, vol. 52, no. 3, 2010.
- [9] D. Amodei et al., "Concrete Problems in AI Safety," arXiv:1606.06565, 2016.
- [10] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, "On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?" in *Proc. ACM FAccT*, 2021.
- [11] L. Ouyang et al., "Training Language Models to Follow Instructions with Human Feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [12] Y. Bai et al., "Constitutional AI: Harmlessness from AI Feedback," arXiv:2212.08073, 2022.
- [13] E. Perez et al., "Red Teaming Language Models with Language Models," in *Proc. EMNLP*, 2022.
- [14] National Institute of Standards and Technology, "Artificial Intelligence Risk Management Framework (AI RMF 1.0)," NIST AI 100-1, 2023.
- [15] European Union, Regulation (EU) 2024/1689 (Artificial Intelligence Act), Article 14: Human oversight, 2024.
- [16] The White House, "Fact Sheet: President Donald J. Trump Inaugurates the Era of Super Intelligence," Sept. 29, 2026.
- [17] N. Bostrom, *Superintelligence: Paths, Dangers, Strategies*. Oxford University Press, 2014.